

Web Programming Review 2

Project: PennyWise – Student Expense Tracker

Batch 1: Shourya Chavan (24BCE0509)

Aarush Singh (24BCE0488)

Shambhavi (24BCE0514)

Prisha Rawat (24BCE2545)

Aishashwi Gupta (24BCB0088)

PennyWise is a web-based student expense tracking application designed to help users efficiently manage and monitor their daily financial activities. The primary objective of the project is to provide a simple, responsive, and real-time platform where students can record transactions, analyze spending patterns, and maintain financial awareness.

The application is built using a **component-based architecture in React**, ensuring modularity and scalability. The user interface is designed with **Tailwind CSS**, providing a clean, modern, and mobile-responsive layout. The system follows a **Single Page Application (SPA)** approach, where navigation between different sections such as Dashboard, Transaction History, and Admin Portal is handled seamlessly using state-based routing without page reloads.

The core functionality of PennyWise includes **CRUD operations (Create, Read, Delete)** for financial transactions. Users can add income and expense records through a validated input form, which ensures that all entries are meaningful and error-free. Each transaction is categorized and stored, allowing structured financial tracking. The application dynamically calculates **Total Balance, Total Income, and Total Expenses** using optimized state management techniques, ensuring real-time updates across the interface.

A key feature of the system is the **Student Dashboard**, which serves as the main interface for users. It displays financial summaries and includes a visual representation of expenses through a **dynamic doughnut chart implemented using the HTML5 Canvas API**. This

helps users quickly understand how their expenses are distributed across different categories.

The **Transaction History module** provides a detailed list of all recorded transactions, along with advanced filtering options based on transaction type and category. This enhances usability by allowing users to analyze specific financial patterns and retrieve relevant data efficiently.

In Review 2, the project integrates **Firebase services**, including Firestore and Authentication, to enable **data persistence and real-time synchronization**. Unlike earlier versions where data was stored temporarily, transactions are now securely stored in the cloud and remain consistent across sessions. Firebase Authentication ensures that each user has a unique and secure session, enabling proper data isolation.

Additionally, the **System Admin Portal** provides an overview of system-level metrics such as database status, total records, global liquidity, and active user sessions. This demonstrates the scalability and administrative capabilities of the application.

Overall, PennyWise evolves from a basic frontend prototype into a **fully functional, real-time expense management system**, showcasing modern web development practices including React state management, cloud integration, dynamic visualization, and responsive UI design.

Main Code -

```

import React, { useState, useMemo, useEffect, useRef } from 'react'; import { initializeApp } from
'firebase/app'; import { getAuth, signInWithCustomToken, signInAnonymously, onAuthStateChanged } from
'firebase/auth'; import { getFirestore, collection, doc, setDoc, deleteDoc, onSnapshot } from
'firebase/firestore';

// --- FIREBASE INITIALIZATION (REVIEW 2 UPGRADE) --- // ⚠️ IMPORTANT: Paste your Firebase config
details inside the {} below! const localFirebaseConfig = { apiKey:
"AlzaSyAyixGilj1EzSsKZrAMvQN63DY1IL4uR_w", authDomain: "pennywise-2ecd2.firebaseio.com",
projectId: "pennywise-2ecd2", storageBucket: "pennywise-2ecd2.firebaseio.com", messagingSenderId:
"717299628205", appId: "1:717299628205:web:b41bef893175cf6accae24", measurementId: "G-
NQC363VJS" };

const firebaseConfig = typeof _firebase_config !== 'undefined' ? JSON.parse(_firebase_config) :
localFirebaseConfig; const isFirebaseConfigured = Object.keys(firebaseConfig).length > 0;

let app, auth, db; if (isFirebaseConfigured) { app = initializeApp(firebaseConfig); auth = getAuth(app); db =
getFirestore(app); } const appId = typeof __app_id !== 'undefined' ? __app_id : 'pennywise-local';

// --- INLINE SVG ICONS (Kept from Review 1) --- const IconPieChart = ({ className }) => (); const IconList =
({ className }) => (); const IconWallet = ({ className }) => (); const IconTrash2 = ({ className }) => (); const
IconPlusCircle = ({ className }) => (); const IconArrowUpRight = ({ className }) => (); const
IconArrowDownRight = ({ className }) => (); const IconUtensils = ({ className }) => (); const IconCar =
({ className }) => (); const IconBookOpen = ({ className }) => (); const IconFilm = ({ className }) => ();
const IconMoreHorizontal = ({ className }) => (); const IconShield = ({ className }) => (); const
IconDatabase = ({ className }) => (); const IconCloud = ({ className }) => ();

// --- MAIN APP COMPONENT --- export default function App() { const [transactions, setTransactions] =
useState([]); const [currentTab, setCurrentTab] = useState('dashboard');

// Review 2: Firebase Auth State const [user, setUser] = useState(null); const [isLoading, setIsLoading] =
useState(true); const [systemError, setSystemError] = useState(""); // Added error tracking

// 1. Firebase Authentication Effect useEffect(() => { if (!isFirebaseConfigured) return;

const initAuth = async () => {
  try {
    if (typeof __initial_auth_token !== 'undefined' && __initial_auth_token) {
      await signInWithCustomToken(auth, __initial_auth_token);
    } else {
      await signInAnonymously(auth);
    }
  } catch (error) {
    console.error("Auth Error:", error);
    setSystemError(Authentication Failed: ${error.message}. Please enable Anonymous

```

```

Sign-in in Firebase.);
    }
};
initAuth();

const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
    setUser(currentUser);
    setIsLoading(false);
});

return () => unsubscribe();

}, []);

// 2. Firestore Data Fetching Effect useEffect(() => { if (!isFirebaseConfigured || !user) return; // Guard clause:
only fetch if logged in

const txnsRef = collection(db, 'artifacts', appId, 'users', user.uid,
'transactions');

// onSnapshot sets up a real-time listener to the database
const unsubscribe = onSnapshot(txnsRef, (snapshot) => {
    const fetchedTxns = [];
    snapshot.forEach((doc) => {
        fetchedTxns.push({ id: doc.id, ...doc.data() });
    });

    // Sort manually by date (newest first) to avoid complex DB indexes for Review 2
    fetchedTxns.sort((a, b) => new Date(b.date) - new Date(a.date));
    setTransactions(fetchedTxns);
}, (error) => {
    console.error("Firestore Listen Error:", error);
});

return () => unsubscribe();

}, [user]);

// 3. Database Core Logic (Review 2 Upgrade) const addTransaction = async (newTxnData) => { if (!user)
{ setSystemError("Cannot save: You are not logged in. Please enable 'Anonymous' Sign-in in Firebase
Authentication."); return; } try { // Create a new document reference with an auto-generated ID const
newDocRef = doc(collection(db, 'artifacts', appId, 'users', user.uid, 'transactions')); await
setDoc(newDocRef, newTxnData); setSystemError(""); // Clear any previous errors on success } catch (error)

```

```
{ console.error("Error adding transaction:", error); setSystemError(Database Error: ${error.message} (Did you start Firestore in 'Test Mode'?)); };
```

```
const deleteTransaction = async (id) => { if (!user) return; try { await deleteDoc(doc(db, 'artifacts', appld, 'users', user.uid, 'transactions', id)); } catch (error) { console.error("Error deleting transaction:", error); } };
```

```
// 4. Derived State Calculations const { totalBalance, totalIncome, totalExpense } = useMemo(() => { let income = 0; let expense = 0; transactions.forEach(txn => { if (txn.type === 'income') income += Number(txn.amount); if (txn.type === 'expense') expense += Number(txn.amount); }); return { totalBalance: income - expense, totalIncome: income, totalExpense: expense }; }, [transactions]);
```

```
// Loading State if (!isFirebaseConfigured) { return (
```

Firestore Not Configured

You are seeing this because the app is trying to connect to a database, but you haven't provided your Firebase API keys yet.

Go to **App.jsx (Line 8)** and replace `localFirebaseConfig` with your actual keys from the Firebase Console.

```
);}
```

```
if (isLoading) { return (
```

```
Connecting to Secure Database...
```

```
);}
```

```
// 5. Render active view based on "Routing" const renderView = () => { switch (currentTab) { case 'dashboard': return ( ); case 'history': return ( ); case 'admin': return ( ); default: return ( ); }
```

```
return (
```

PennyWise v2.0 (Review 2)

```
<button onClick={() => setCurrentTab('dashboard')} className={px-3 py-2 rounded-md text-sm font-medium transition-colors flex items-center gap-2 ${currentTab === 'dashboard' ? 'bg-indigo-700 text-white': 'text-indigo-100 hover:bg-indigo-500'}}> Dashboard <button onClick={() => setCurrentTab('history')} className={px-3 py-2 rounded-md text-sm font-medium transition-colors flex items-center gap-2 ${currentTab === 'history' ? 'bg-indigo-700 text-white': 'text-indigo-100 hover:bg-indigo-500'}}> History <button onClick={() => setCurrentTab('admin')} className={px-3 py-2 rounded-md text-sm font-medium transition-colors flex items-center gap-2 ${currentTab === 'admin' ? 'bg-slate-800 text-emerald-400': 'text-indigo-100 hover:bg-indigo-500'}}> Admin <main className="max-w-4xl mx-auto p-4 sm:p-6 lg:p-8 animate-fade-in"> {systemError && ( <div className="mb-6 bg-red-50 border border-red-200 text-red-700 p-4 rounded-xl flex items-start gap-3">
```

```

        <IconShield className="w-6 h-6 flex-shrink-0 mt-0.5" />
        <div>
          <h3 className="font-bold">System Connection Error</h3>
          <p className="text-sm mt-1">{systemError}</p>
        </div>
      </div>
    )}
    {renderView()}
  </main>
</div>

```

```
);}
```

```

// --- DASHBOARD COMPONENT --- function Dashboard({ totalBalance, totalIncome, totalExpense,
onAddTransaction, transactions }) { return (

```

Student Dashboard

Track your campus expenses in real-time.

```

{ /* Balance Cards */ }
<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
  <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 flex
flex-col justify-center">
    <span className="text-gray-500 text-sm font-medium">Total Balance</span>
    <span className={text-3xl font-bold mt-2 ${totalBalance < 0 ? 'text-red-500' :
'text-gray-900'}}>
      ₹{totalBalance.toLocaleString()}
    </span>
  </div>
  <div className="bg-emerald-50 p-6 rounded-2xl shadow-sm border border-emerald-100
flex items-center justify-between">
    <div>
      <span className="text-emerald-700 text-sm font-medium">Income</span>
      <div className="text-2xl font-bold text-emerald-600 mt-
2">₹{totalIncome.toLocaleString()}</div>
    </div>
    <div className="bg-emerald-200 p-3 rounded-full text-emerald-700">
      <IconArrowUpRight className="w-6 h-6" />
    </div>
  </div>
  <div className="bg-rose-50 p-6 rounded-2xl shadow-sm border border-rose-100 flex
items-center justify-between">
    <div>

```

```

        <span className="text-rose-700 text-sm font-medium">Expenses</span>
        <div className="text-2xl font-bold text-rose-600 mt-2">₹{totalExpense.toLocaleString()}</div>
      </div>
      <div className="bg-rose-200 p-3 rounded-full text-rose-700">
        <IconArrowDownRight className="w-6 h-6" />
      </div>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 items-start">
    { /* HTML5 Canvas Chart - ACTIVE FOR REVIEW 2 */ }
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 flex flex-col items-center text-center h-[400px]">
      <h2 className="text-lg font-bold text-gray-800 mb-4 self-start">Expense Breakdown</h2>
      <div className="flex-1 w-full flex items-center justify-center">
        <ExpenseChart transactions={transactions} totalExpense={totalExpense} />
      </div>
    </div>

    { /* Transaction Form */ }
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 h-[400px]">
      <h2 className="text-lg font-bold text-gray-800 mb-4">Add Transaction</h2>
      <TransactionForm onAdd={onAddTransaction} />
    </div>
  </div>
</div>

);}

// --- HTML5 CANVAS CHART COMPONENT (REVIEW 2 UPGRADE) --- function ExpenseChart({ transactions, totalExpense }) { const canvasRef = useRef(null);

useEffect(() => { const canvas = canvasRef.current; if (!canvas) return; const ctx = canvas.getContext('2d'); const width = canvas.width; const height = canvas.height; const radius = Math.min(width, height) / 2 - 20; // 20px padding

// Clear previous drawing
ctx.clearRect(0, 0, width, height);

if (totalExpense === 0) {
  // Empty State Circle
  ctx.beginPath();

```

```

    ctx.arc(width / 2, height / 2, radius, 0, 2 * Math.PI);
    ctx.fillStyle = '#f3f4f6'; // gray-100
    ctx.fill();
    ctx.lineWidth = 2;
    ctx.strokeStyle = '#e5e7eb'; // gray-200
    ctx.stroke();
    ctx.fillStyle = '#9ca3af'; // gray-400
    ctx.font = '14px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText('No expenses yet', width / 2, height / 2);
    return;
}

// Calculate totals per category
const categoryTotals = {};
transactions.forEach(txn => {
  if (txn.type === 'expense') {
    categoryTotals[txn.category] = (categoryTotals[txn.category] || 0) + txn.amount;
  }
});

// Colors mapping
const colorMap = {
  'Food': '#f59e0b',      // Amber
  'Travel': '#3b82f6',    // Blue
  'Academics': '#8b5cf6', // Purple
  'Entertainment': '#ec4899', // Pink
  'Other': '#64748b'      // Slate
};

let startAngle = -0.5 * Math.PI; // Start at top (12 o'clock)

Object.keys(categoryTotals).forEach(category => {
  const amount = categoryTotals[category];
  const sliceAngle = (amount / totalExpense) * 2 * Math.PI;

  // Draw Slice
  ctx.beginPath();
  ctx.moveTo(width / 2, height / 2);
  ctx.arc(width / 2, height / 2, radius, startAngle, startAngle + sliceAngle);
  ctx.closePath();
  ctx.fillStyle = colorMap[category] || colorMap['Other'];
  ctx.fill();

  // Outline for slice separation
  ctx.lineWidth = 2;

```



```

    ctx.strokeStyle = 'ffffff';
    ctx.stroke();

    startAngle += sliceAngle;
  });

  // Draw inner circle for "Doughnut" chart look
  ctx.beginPath();
  ctx.arc(width / 2, height / 2, radius * 0.6, 0, 2 * Math.PI);
  ctx.fillStyle = 'ffffff';
  ctx.fill();

  // Add Total text in the middle
  ctx.fillStyle = '#374151'; // gray-700
  ctx.font = 'bold 20px sans-serif';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText('₹' + totalExpense.toLocaleString(), width / 2, height / 2);

}, [transactions, totalExpense]);

return (

  /* Legend */
  {totalExpense > 0 && (
    <div className="flex flex-wrap justify-center gap-3 mt-4 px-2">
      {[ 'Food', 'Travel', 'Academics', 'Entertainment', 'Other' ].map(cat => {
        const hasData = transactions.some(t => t.type === 'expense' && t.category ===
cat);
        if (!hasData) return null;
        const colors = { 'Food': 'bg-amber-500', 'Travel': 'bg-blue-500', 'Academics':
'bg-purple-500', 'Entertainment': 'bg-pink-500', 'Other': 'bg-slate-500' };
        return (
          <div key={cat} className="flex items-center gap-1 text-xs text-gray-600">
            <span className="w-3 h-3 rounded-full ${colors[cat]}></span> {cat}
          </div>
        );
      })}
    </div>
  )}
);
// --- TRANSACTION FORM COMPONENT --- function TransactionForm({ onAdd }) { const [title, setTitle] =
useState(""); const [amount, setAmount] = useState(""); const [type, setType] = useState('expense'); const

```

```

[category, setCategory] = useState('Food'); const [date, setDate] = useState(new
Date().toISOString().split('T')[0]); const [error, setError] = useState("");

const expenseCategories = ['Food', 'Travel', 'Academics', 'Entertainment', 'Other']; const incomeCategories =
['Allowance', 'Freelance', 'Prize Money', 'Other'];

const handleSubmit = (e) => { e.preventDefault(); setError("");

if (!title.trim() || !amount) {
  setError('Please fill in all required fields. ');
  return;
}
if (Number(amount) <= 0) {
  setError('Amount must be greater than zero. ');
  return;
}

const newTxn = {
  title: title.trim(),
  amount: Number(amount),
  type,
  category,
  date
};

onAdd(newTxn);
setTitle('');
setAmount('');

};

return (

{error &&
{error}
}
<div className="flex bg-gray-100 p-1 rounded-lg">
  <button type="button" onClick={() => { setType('expense');
setCategory('Food'); }} className={flex-1 py-1.5 text-sm font-medium rounded-md
transition-all ${type === 'expense' ? 'bg-white text-gray-900 shadow' : 'text-gray-
500 hover:text-gray-700'}}>Expense</button>
  <button type="button" onClick={() => { setType('income');
setCategory('Allowance'); }} className={flex-1 py-1.5 text-sm font-medium rounded-md
transition-all ${type === 'income' ? 'bg-white text-gray-900 shadow' : 'text-gray-500
hover:text-gray-700'}}>Income</button>
</div>

```

```

<div className="grid grid-cols-2 gap-3">
  <div className="col-span-2">
    <input type="text" value={title} onChange={(e) => setTitle(e.target.value)}
placeholder="Description (e.g. Library Fine)" className="w-full px-3 py-2 text-sm bg-
gray-50 border border-gray-200 rounded focus:outline-none focus:ring-1 focus:ring-
indigo-500" />
  </div>
  <div>
    <input type="number" value={amount} onChange={(e) => setAmount(e.target.value)}
placeholder="Amount (₹)" className="w-full px-3 py-2 text-sm bg-gray-50 border
border-gray-200 rounded focus:outline-none focus:ring-1 focus:ring-indigo-500" />
  </div>
  <div>
    <input type="date" value={date} onChange={(e) => setDate(e.target.value)}
className="w-full px-3 py-2 text-sm bg-gray-50 border border-gray-200 rounded
focus:outline-none focus:ring-1 focus:ring-indigo-500 text-gray-600" />
  </div>
</div>

<div>
  <select value={category} onChange={(e) => setCategory(e.target.value)}
className="w-full px-3 py-2 text-sm bg-gray-50 border border-gray-200 rounded
focus:outline-none focus:ring-1 focus:ring-indigo-500 text-gray-700">
    {type === 'expense'
      ? expenseCategories.map(cat => <option key={cat} value={cat}>{cat}</option>)
      : incomeCategories.map(cat => <option key={cat} value={cat}>{cat}</option>)
    }
  </select>
</div>

<button type="submit" className="w-full flex items-center justify-center gap-2 py-2
bg-indigo-600 text-white rounded font-medium hover:bg-indigo-700 transition-colors
text-sm">
  <IconPlusCircle className="w-4 h-4" /> Save to Database
</button>
</form>

```

```
);}
```

```

// --- HISTORY / LIST COMPONENT (REVIEW 2 UPGRADE: FILTERS ADDED) --- function History({ transactions,
onDelete }) { const [filterType, setFilterType] = useState('All'); const [filterCategory, setFilterCategory] =
useState('All');

```

```

const handleTypeChange = (type) => { setFilterType(type); setFilterCategory('All'); // Reset category when
switching types so we don't get stuck on empty lists };

const getCategoryIcon = (category, type) => { if (type === 'income') return ; switch (category) { case 'Food':
return ; case 'Travel': return ; case 'Academics': return ; case 'Entertainment': return ; default: return ; }};

// Get dynamic categories based on current type filter const availableCategories = ['All', ...new
Set(transactions .filter(txn => filterType === 'All' || (filterType === 'Income' ? txn.type === 'income' : txn.type
=== 'expense')) .map(t => t.category) )];

// Filter Logic const filteredTransactions = transactions.filter(txn => { const matchType = filterType === 'All' ||
(filterType === 'Income' && txn.type === 'income') || (filterType === 'Expense' && txn.type === 'expense');
const matchCategory = filterCategory === 'All' || txn.category === filterCategory; return matchType &&
matchCategory; });

return (

```

Transaction History

Review and filter your financial records.

```

{/* Filtering Controls */}
<div className="flex flex-col sm:flex-row gap-3 self-start sm:self-auto w-full
sm:w-auto">
  <div className="flex bg-gray-100 p-1 rounded-lg">
    {[ 'All', 'Income', 'Expense' ].map(type => (
      <button key={type}
        onClick={() => handleTypeChange(type)}
        className={px-4 py-1.5 text-sm font-medium rounded-md transition-all
${filterType === type ? 'bg-white text-gray-900 shadow' : 'text-gray-500 hover:text-
gray-700'}}
      >
        {type}
      </button>
    ))}
  </div>

  <select
    value={filterCategory}
    onChange={(e) => setFilterCategory(e.target.value)}
    className="px-3 py-1.5 text-sm bg-gray-100 border border-transparent
hover:border-gray-200 rounded-lg focus:outline-none focus:ring-2 focus:ring-indigo-
500 text-gray-700 font-medium cursor-pointer transition-colors outline-none"
  >
    {availableCategories.map(cat => (

```

```

        <option key={cat} value={cat}>>{cat === 'All' ? 'All Categories' :
cat}</option>
      )}}
    </select>
  </div>
</header>

<div className="bg-white rounded-2xl shadow-sm border border-gray-100 overflow-
hidden">
  {filteredTransactions.length === 0 ? (
    <div className="p-8 text-center text-gray-500 flex flex-col items-center">
      <IconList className="w-12 h-12 text-gray-300 mb-3" />
      <p className="font-medium">No matching transactions found.</p>
    </div>
  ) : (
    <ul className="divide-y divide-gray-100">
      {filteredTransactions.map(txn => (
        <li key={txn.id} className="p-4 sm:p-6 hover:bg-gray-50 transition-colors
flex items-center justify-between group">
          <div className="flex items-center gap-4">
            <div className={p-3 rounded-full ${txn.type === 'income' ? 'bg-emerald-
100' : 'bg-gray-100'}}>
              {getCategoryIcon(txn.category, txn.type)}
            </div>
            <div>
              <h3 className="font-semibold text-gray-900">{txn.title}</h3>
              <div className="flex items-center gap-2 mt-1 text-xs text-gray-500">
                <span>{new Date(txn.date).toLocaleDateString('en-GB', { day:
'numeric', month: 'short', year: 'numeric' })}</span>
                <span>•</span>
                <span className="px-2 py-0.5 bg-gray-100 rounded-
full">{txn.category}</span>
              </div>
            </div>
          </div>

          <div className="flex items-center gap-4">
            <span className={font-bold ${txn.type === 'income' ? 'text-emerald-
600' : 'text-gray-900'}}>
              {txn.type === 'income' ? '+' : '-'}₹{txn.amount.toLocaleString()}
            </span>
            <button onClick={() => onDelete(txn.id)} className="p-2 text-gray-400
hover:text-red-500 hover:bg-red-50 rounded-lg transition-colors opacity-100
sm:opacity-0 sm:group-hover:opacity-100 focus:opacity-100" title="Delete
Transaction">
              <IconTrash2 className="w-4 h-4" />
            </button>
          </div>
        </li>
      )

```

```

        </div>
      </li>
    )})
  </ul>
})
</div>
</div>

```

```
);}
```

```
// --- ADMIN PANEL COMPONENT (REVIEW 2 UPGRADE) --- function AdminPanel({ transactions,
totalBalance, user }) { return (
```

System Admin Portal

Platform overview and active connections.

```

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  { /* Real Firebase Status */ }
  <div className="bg-white p-6 rounded-xl border border-emerald-200 shadow-sm">
    <div className="flex items-center gap-2 mb-4 text-gray-700 font-semibold">
      <IconDatabase className="w-5 h-5 text-emerald-500" />
      Database Status
    </div>
    <div className="space-y-3">
      <div className="flex justify-between text-sm">
        <span className="text-gray-500">Active Connection:</span>
        <span className="font-medium text-emerald-700 bg-emerald-50 px-2 py-0.5
rounded flex items-center gap-1">
          <span className="w-2 h-2 rounded-full bg-emerald-500 animate-
pulse"></span> Firebase Firestore
        </span>
      </div>
      <div className="flex justify-between text-sm">
        <span className="text-gray-500">Document Count:</span>
        <span className="font-medium text-gray-900">{transactions.length}
Records</span>
      </div>
      <div className="flex justify-between text-sm">
        <span className="text-gray-500">Global Liquidity:</span>
        <span className={font-medium ${totalBalance < 0 ? 'text-red-600' : 'text-
emerald-600'}}>₹{totalBalance.toLocaleString()}</span>
      </div>
    </div>
  </div>

```

```

    <div className="mt-6 pt-4 border-t border-gray-100">
      <p className="text-xs text-gray-500 leading-relaxed">
        * Note: For Review 2, data is now successfully persisting to the cloud via
        Firebase. Local React State has been upgraded to real-time snapshot listeners.
      </p>
    </div>
  </div>

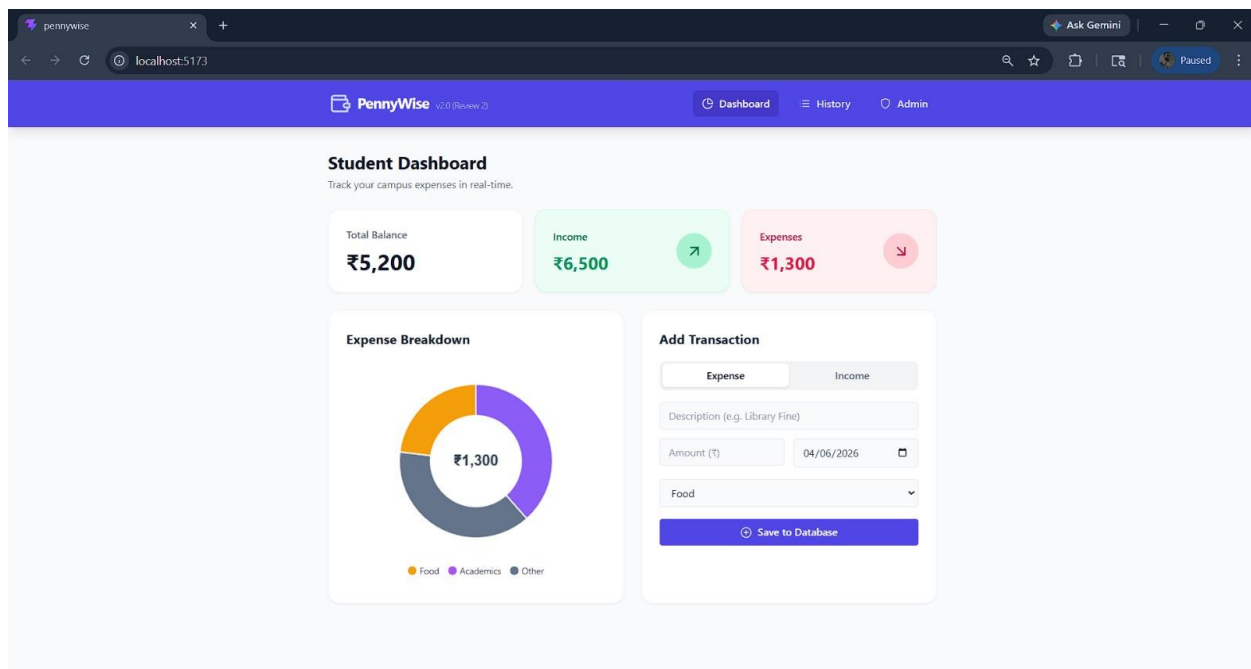
  { /* Real User Profile */ }
  <div className="bg-slate-800 p-6 rounded-xl border border-slate-700 shadow-sm
  text-white">
    <div className="flex items-center gap-2 mb-4 text-slate-200 font-semibold">
      <IconList className="w-5 h-5 text-emerald-400" />
      Active User Session
    </div>
    <div className="bg-slate-900/50 rounded-lg p-3 border border-slate-700 mb-3
  flex flex-col gap-2">
      <div className="flex items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="w-8 h-8 rounded-full bg-indigo-500 flex items-center
  justify-center font-bold text-sm">U1</div>
          <div>
            <div className="text-sm font-medium">Verified Student</div>
            <div className="text-xs text-slate-400 font-mono mt-0.5">{user?.uid ?
  user.uid.substring(0, 12) : 'Not Logged In'}...</div>
          </div>
        </div>
        <div className="text-xs font-mono text-emerald-400 bg-emerald-400/10 px-2
  py-1 rounded">Online</div>
      </div>
    </div>

    <p className="text-xs text-slate-400 leading-relaxed mt-4">
      Firebase Auth has assigned a secure, unique UID to the current session,
      ensuring complete data isolation in Firestore.
    </p>
  </div>
</div>
);}

```

Our Website -

Student Dashboard -



The **Student Dashboard** acts as the central interface for monitoring financial activity, similar to Review 1, but enhanced with real-time data and visualization features.

In Review 2, the dashboard dynamically displays **Total Balance, Income, and Expenses**, computed using React's `useMemo` hook for optimized performance. These values automatically update whenever transaction data changes, ensuring efficient re-rendering.

A major enhancement in this version is the integration of an **Expense Breakdown visualization using the HTML5 Canvas API**. Instead of a placeholder, the system now renders a **dynamic doughnut chart**, where each segment represents a category of expenses (e.g., Food, Academics, Travel). The chart recalculates proportions based on live transaction data.

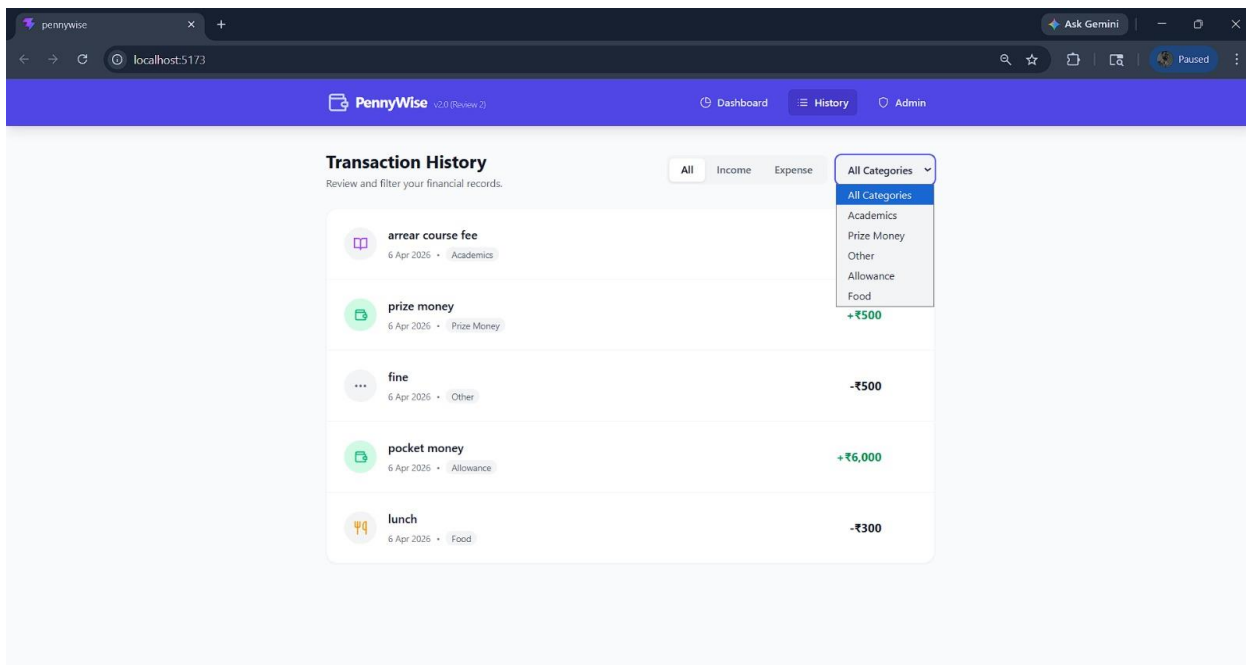
Additionally, the **Add Transaction form** has been retained and improved. It performs:

- Input validation (empty fields, invalid amounts)
- Dynamic category switching based on transaction type (Income/Expense)

Upon submission, the data is now stored in a **cloud database (Firebase Firestore)** rather than local state, enabling persistence and real-time synchronization.

This ensures that all updates are reflected instantly without requiring a page reload, improving both performance and user experience.

Transaction History -



The **Transaction History** module provides a detailed and interactive view of all financial records.

In Review 2, this component has been significantly enhanced with **advanced filtering capabilities**, which were previously deferred:

- Users can filter transactions by **type** (All, Income, Expense)
- A dropdown allows filtering by **category** (e.g., Food, Academics, Prize Money)

The filtering logic dynamically processes the transactions array and displays only the matching results, improving usability and data accessibility.

Each transaction entry includes:

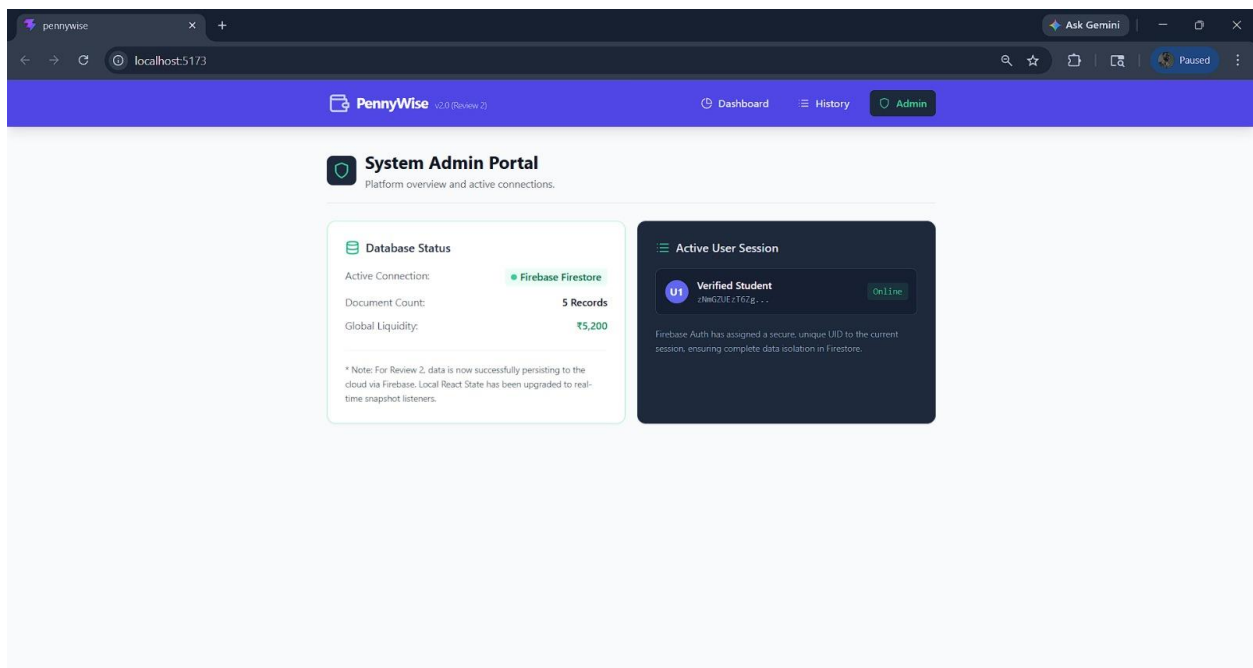
- Title

- Date (formatted dynamically)
- Category tag
- Amount (with conditional formatting: green for income, neutral/red for expense)

The delete functionality is preserved, allowing users to remove entries, with changes immediately reflected due to real-time database updates.

This module now demonstrates a complete **read + filter + delete system**, making it far more interactive compared to Review 1.

System Admin Portal -



The **System Admin Portal** has been upgraded from a static MVP view to a more realistic system monitoring interface.

In Review 2, the panel now reflects **live backend integration** instead of local state. It displays:

- Active database connection (Firebase Firestore)
- Total number of stored records (fetched from database)

- Global liquidity (computed from all transactions)

A new addition is the **Active User Session panel**, which uses Firebase Authentication to:

- Identify the current user
- Display a unique user ID (UID)
- Indicate real-time session status (Online)

This demonstrates secure session handling and proper user isolation in a cloud environment.

Unlike Review 1's mock data, this version reflects actual backend-driven data, showing a transition from a prototype to a more production-ready system.

Navigation and Routing Logic -

The application continues to use a **state-based routing mechanism**, controlled by the `currentTab` variable.

Based on user interaction with the navigation bar:

- Dashboard view is rendered
- History view is rendered
- Admin panel is rendered

This approach maintains a **Single Page Application (SPA)** structure, ensuring smooth transitions without page reloads.

In Review 2, this routing system integrates seamlessly with:

- Real-time database updates
- Authentication state changes

This results in a more responsive and dynamic user experience compared to Review 1.